AIC Controller

Mathematical Foundations & Algorithm Reference

Auto-generated from source — `aic_controller` package

March 25, 2026

Contents

1 Architecture Overview

The AIC Controller is a ROS 2 `controller_interface::ControllerInterface` plugin. It runs inside `ros2_control` at a fixed frequency and supports two *control modes*:

Mode	Output Interface	Status
Impedance	Joint effort (torque)	Implemented
Admittance	Joint position	Not yet implemented

Within impedance mode, two *target modes* select the coordinate space of incoming commands:

Target Mode	Message Type	Impedance Action
Cartesian	`MotionUpdate`	`CartesianImpedanceAction`
Joint	`JointMotionUpdate`	`JointImpedanceAction`

Each control cycle executes:

Read Hardware → Ingest Commands → Generate Reference → Compute Torques → Write Hardware

2 Mathematical Preliminaries

2.1 Rigid Body Poses — $\mathrm{SE}(3)$ and $\mathrm{SO}(3)$

A rigid body pose in 3D is an element of the *Special Euclidean group*:

$$T = \begin{bmatrix} R & p \\ \mathbf{0}^\mathsf{T} & 1 \end{bmatrix} \in \mathrm{SE}(3), \qquad R \in \mathrm{SO}(3),\ p \in \mathbb{R}^3$$

where R is a rotation matrix and p is a translation vector.

In code, poses are stored as `Eigen::Isometry3d` — `pose.linear()` gives R and `pose.translation()` gives p.

2.2 Exponential and Logarithmic Maps

The Lie algebra $\mathfrak{so}(3)$ consists of 3×3 skew-symmetric matrices. The *hat* operator maps $\mathbb{R}^3 \to \mathfrak{so}(3)$:

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} \mapsto [\boldsymbol{\omega}]_\times = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}$$

Exponential map (tangent vector → rotation):

$$\mathrm{Exp} : \mathbb{R}^3 \to \mathrm{SO}(3), \qquad R = \exp([\boldsymbol{\omega}]_\times)$$

Computed via the Rodrigues formula. For quaternions (Sophus `SO3::exp`):

$$q = \mathrm{Exp}(\boldsymbol{\delta}), \qquad \theta = \|\boldsymbol{\delta}\|, \quad q = \left(\cos\tfrac{\theta}{2},\ \tfrac{\boldsymbol{\delta}}{\theta}\sin\tfrac{\theta}{2}\right)$$

Logarithmic map (rotation → tangent vector):

$$\mathrm{Log} : \mathrm{SO}(3) \to \mathbb{R}^3, \qquad \boldsymbol{\omega} = \mathrm{Log}(R)$$

This is the inverse of Exp. Given a unit quaternion q, it extracts the axis-angle vector $\boldsymbol{\omega}$ with $\|\boldsymbol{\omega}\| = \theta$ (rotation angle) and $\boldsymbol{\omega}/\theta$ the rotation axis.

SE(3) integration. The twist is $\boldsymbol{\xi} = [v^\mathsf{T}, \omega^\mathsf{T}]^\mathsf{T} \in \mathbb{R}^6$. Pose integration uses:

SE(3) Integration

$$T_{k+1} = T_k \cdot \mathrm{Exp}(\boldsymbol{\xi}\, \Delta t)$$

Implemented in `utils::integrate_pose` via Sophus `SE3::exp`.

Why Lie groups instead of Euler angles?

Classical Euler angles suffer from gimbal lock and representation singularities. Working directly on SE(3)/SO(3) ensures singularity-free pose integration and consistent orientation error computation.

2.3 Robot Jacobian

For an n-DOF robot, the geometric Jacobian $J(q) \in \mathbb{R}^{6\times n}$ maps joint velocities to the end-effector twist:

$$\boldsymbol{\xi} = J(q)\,\dot{q}$$

where $\boldsymbol{\xi} = [v^\mathsf{T}, \omega^\mathsf{T}]^\mathsf{T}$ is the 6D spatial velocity (linear + angular) of the tool frame.

The Jacobian is computed externally by a `kinematics_interface` plugin (typically KDL- or Pinocchio-based) via `calculate_jacobian()`.

3 Cartesian Impedance Control

3.1 The Spring–Damper Analogy

Impedance control makes the robot behave like a mechanical spring–damper system in Cartesian space. A 1D mass–spring–damper satisfies:

$$m\ddot{x} + d\dot{x} + kx = f_{\text{ext}}$$

In impedance control we *generate* the wrench:

$$F = K \cdot e_x + D \cdot e_v$$

where e_x is position error and e_v is velocity error, allowing compliant interaction with the environment.

3.2 Control Wrench Law

The full 6D control wrench in the base frame is:

Cartesian Impedance Wrench

$$F_{\text{ctrl}} = K\, e_x + D\, e_v + F_{\text{ff}} + K_I \odot \int e_x\, dt + F_{\text{offset}} \tag{1}$$

Symbol	Size	Description
K	6×6	Cartesian stiffness (diagonal)
D	6×6	Cartesian damping (diagonal)
e_x	6×1	Pose error $= x_{\text{des}} - x_{\text{cur}}$
e_v	6×1	Velocity error $= \dot{x}_{\text{des}} - \dot{x}_{\text{cur}}$
F_{ff}	6×1	Feedforward wrench (see §10)
K_I	6×1	Integrator gain (element-wise)
F_{offset}	6×1	Constant offset (e.g. payload)

The pose error e_x is computed by the kinematics plugin's `calculate_frame_difference()`, yielding $[e_{\text{trans}}^{\mathsf{T}}, e_{\text{rot}}^{\mathsf{T}}]^{\mathsf{T}}$ where the rotational part is in axis-angle form.

Example

Suppose TCP position $p = [0.5,\ 0,\ 0.3]\,\text{m}$ and desired $p_d = [0.5,\ 0.05,\ 0.3]\,\text{m}$ with $K = \text{diag}(500, 500, 500, 20, 20, 20)\,\text{N/m}\,(\text{N m/rad})$:

$$e_x = [0,\ 0.05,\ 0,\ 0,\ 0,\ 0]^{\mathsf{T}} \implies F = K\,e_x = [0,\ 25,\ 0,\ 0,\ 0,\ 0]^{\mathsf{T}}\ \text{N}$$

A 25 N restoring force along y.

3.3 PID-Like Extension — Integral Term

An integral term accumulates pose error to eliminate steady-state offset:

$$e_I(k{+}1) = \text{clamp}\big(e_I(k) + e_x(k),\ -B,\ B\big) \tag{2}$$

$$F_I = K_I \odot e_I \tag{3}$$

where $B \in \mathbb{R}^6$ is the integrator bound (anti-windup) and $\odot$ denotes element-wise multiplication.

Why anti-windup?

Without the bound B, large transient errors (e.g. during collisions) cause the integrator to *wind up*, producing dangerous persistent forces even after the error is resolved.

3.4 Mapping Wrench to Joint Torques

The Cartesian wrench is mapped to joint torques via the Jacobian transpose:

Jacobian Transpose Mapping

$$\tau = J^{\mathsf{T}}\,F_{\text{ctrl}} \tag{4}$$

Derivation from virtual work. A virtual joint displacement δq produces a Cartesian displacement $\delta x = J\,\delta q$. The work done by force F is:

$$\delta W = F^{\mathsf{T}}\delta x = F^{\mathsf{T}} J\,\delta q = (J^{\mathsf{T}}F)^{\mathsf{T}}\delta q$$

So the equivalent joint torque is $\tau = J^{\mathsf{T}}F$. This does *not* require inverting the Jacobian and works even at singularities (though Cartesian stiffness degenerates in singular directions).

3.5 Wrench and Torque Saturation

Before applying J^T, each wrench component is clamped:

$$F_{\text{ctrl},i} = \text{clamp}(F_{\text{ctrl},i},\ -F_{\max,i},\ F_{\max,i})$$

After computing all joint torque contributions, each joint torque is clamped to the URDF effort limit:

$$\tau_i = \text{clamp}(\tau_i,\ -\tau_{\max,i},\ \tau_{\max,i})$$

4 Nullspace Control

4.1 Redundancy and the Nullspace

A robot with $n > 6$ joints has *kinematic redundancy*. The extra DOFs span the nullspace of J:

$$\mathcal{N}(J) = \{\, \dot{q} \in \mathbb{R}^n \mid J\,\dot{q} = \mathbf{0} \,\}$$

Nullspace control adds torques that move joints *without* affecting the TCP, useful for preferred-posture regulation, joint-limit avoidance, or manipulability optimisation.

4.2 Nullspace Projector

The nullspace projection matrix is:

$$N = I_n - J\,J^\dagger \tag{5}$$

where $J^\dagger$ is the right pseudo-inverse of J. Any torque $\tau_0 \in \mathbb{R}^n$ projected through N produces zero TCP acceleration:

$$\tau_{\text{ns}} = N\,\tau_0$$

Proof sketch. $J\,J^\dagger\,J = J$, so $J\,N = J(I - JJ^\dagger) = J - J = \mathbf{0}$. Therefore the Cartesian acceleration produced by τ_{ns} is zero in the linearised dynamics.

4.3 Singularity-Robust Pseudo-Inverse (SVD Regularisation)

The naïve pseudo-inverse $J^\dagger = J^\mathsf{T}(JJ^\mathsf{T})^{-1}$ diverges near singularities. This implementation uses a *damped pseudo-inverse* following **Chiaverini (1997)** with per-dimension adaptive damping.

Compute the SVD of $M = JJ^\mathsf{T}$:

$$M = U\,\Sigma\,V^\mathsf{T}, \qquad \Sigma = \text{diag}(\sigma_1,\,\sigma_2,\,\ldots,\,\sigma_m), \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_m \geq 0$$

The regularised inverse diagonal entries are:

Regularised Singular Values

$$\Sigma^\dagger_{ii} = \frac{\sigma_i}{\sigma_i^2 + \lambda_i^2} \tag{6}$$

The per-dimension condition number is:

$$\text{cn}_i = \frac{\sigma_1}{\sigma_i + 10^{-10}}$$

The adaptive damping factor is:

$$\lambda_i^2 = \begin{cases} \left(1 - \left(\frac{c_{\text{th}}}{\text{cn}_i}\right)^2\right)^2 \cdot \frac{1}{\epsilon} & \text{if } \text{cn}_i > c_{\text{th}} \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

where $c_{\text{th}} = 1500$ is the condition number threshold and $\epsilon = 10^{-6}$.

The bisquare function $f(r) = (1 - r^2)^2$.

- $f(0) = 1$: full damping when condition number is extremely high
- $f(1) = 0$: zero damping at the threshold boundary
- $f'(0) = 0$ and $f'(1) = 0$: smooth onset/offset — no discontinuous jumps

The final pseudo-inverse is:

$$J^\dagger = J^\mathsf{T} V \Sigma^\dagger U^\mathsf{T} \tag{8}$$

Singularity example

Consider a 7-DOF arm near a shoulder singularity where $\sigma_6 \approx 0.001$ and $\sigma_1 = 5.0$:

$$\text{cn}_6 = \frac{5.0}{0.001} = 5000 \gg 1500 \qquad \Longrightarrow \qquad \lambda_6^2 = \left(1 - (1500/5000)^2\right)^2 \cdot 10^6 \approx 7.6 \times 10^5$$

This large λ_6^2 suppresses the near-zero singular value, preventing torque spikes.

4.4 Nullspace Impedance Torque

The raw nullspace torque is a joint-space PD controller:

$$\tau_0 = K_{\text{ns}} \odot (q_{\text{ns}}^* - q) - D_{\text{ns}} \odot \dot{q} \tag{9}$$

where q_{ns}^* is the desired nullspace configuration and K_{ns}, $D_{\text{ns}} \in \mathbb{R}^n$ are per-joint stiffness/damping.

Projected through the nullspace:

$$\tau_{\text{ns}} = N \tau_0 = (I - J J^\dagger) \tau_0 \tag{10}$$

5 Joint-Limit Avoidance

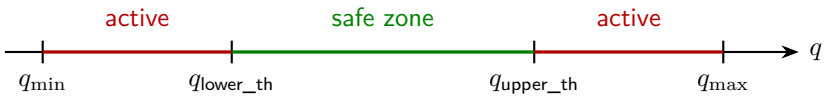
5.1 Activation Zone

For each joint k, an activation zone is defined as a percentage $\alpha \in (0, 1]$ of the joint range:

$$q_{\text{range}} = q_{\text{max}} - q_{\text{min}}$$

$$q_{\text{upper\_th}} = q_{\text{min}} + \frac{\alpha\, q_{\text{range}}}{2} \tag{11}$$

$$q_{\text{lower\_th}} = q_{\text{max}} - \frac{\alpha\, q_{\text{range}}}{2} \tag{12}$$



5.2 Potential Field Torque

Within the activation zone, a linear repulsive potential pushes the joint away from its limit:

$$\tau_{\text{avoid},k} = \begin{cases} g_u \cdot (q_{\text{upper\_th}} - q_k) & \text{if } q_k > q_{\text{upper\_th}} \\ g_l \cdot (q_{\text{lower\_th}} - q_k) & \text{if } q_k < q_{\text{lower\_th}} \\ 0 & \text{otherwise} \end{cases} \tag{13}$$

The gains are chosen so the torque reaches $\tau_{\max}$ exactly at the limit:

$$g_u = \frac{\tau_{\max}}{|q_{\max} - q_{\text{upper\_th}}|}, \qquad g_l = \frac{\tau_{\max}}{|q_{\min} - q_{\text{lower\_th}}|} \tag{14}$$

If the joint exceeds its limit, the torque is *saturated* at $\tau_{\max}$.

6 Joint-Space Impedance Control

For joint-target mode, a diagonal impedance law is used:

Joint Impedance Law

$$\tau = K \odot (q^* - q) + D \odot (\dot{q}^* - \dot{q}) + \tau_{\text{ff}} \tag{15}$$

where K, $D \in \mathbb{R}^n$ are per-joint stiffness/damping vectors, q^*, $\dot{q}^*$ are reference position/velocity, τ_{ff} is feedforward torque, and all operations are element-wise.

Each torque is clamped: $\tau_i = \text{clamp}(\tau_i, -\tau_{\max,i}, \tau_{\max,i})$.

This mode is simpler than Cartesian impedance — it operates directly in joint space, requiring no Jacobian, no nullspace projection, and no FK-based error computation.

7 Gravity Compensation

The gravity torque vector $\tau_g(q)$ compensates for gravitational forces on robot links:

$$\tau_g(q) = \sum_{i=1}^{n} J_{v,i}^{\mathsf{T}}(q)\, m_i\, \mathbf{g} \tag{16}$$

where $J_{v,i}$ is the linear Jacobian of the CoM of link i, m_i is its mass, and $\mathbf{g} = [0,\ 0,\ -9.80665]^{\mathsf{T}}$ m/s^2.

Computed by KDL's `ChainDynParam::JntToGravity` from the URDF kinematic chain. The result is *added* to the impedance torques:

$$\tau_{\text{total}} = \tau_{\text{impedance}} + \tau_g(q)$$

Reasoning

On real hardware, motor controllers receive raw torque commands, so the controller must explicitly cancel gravity. In simulation the physics engine handles this, so gravity compensation can be disabled.

8 Reference Generation and Interpolation

The controller receives targets asynchronously at a lower frequency and must output smooth references at the control rate.

8.1 Position Mode — SLERP and Linear Interpolation

When $t_{\text{rem}} > 0$ (remaining time to target):

Translation (per-cycle linear interpolation):

$$p_{\text{ref}}(k{+}1) = p_{\text{ref}}(k) + \frac{p_{\text{target}} - p_{\text{ref}}(k)}{f_c \cdot t_{\text{rem}}} \tag{17}$$

where f_c is the control frequency (Hz).

Rotation (SLERP — Spherical Linear Interpolation):

$$q_{\text{ref}}(k{+}1) = \text{SLERP}\left(q_{\text{ref}}(k),\ q_{\text{target}},\ \frac{1}{f_c \cdot t_{\text{rem}}}\right) \tag{18}$$

SLERP interpolates along the great arc on the unit quaternion sphere:

$$\text{SLERP}(q_0,\, q_1,\, t) = \frac{\sin((1-t)\,\Omega)}{\sin\Omega}\, q_0 + \frac{\sin(t\,\Omega)}{\sin\Omega}\, q_1, \qquad \Omega = \arccos(q_0 \cdot q_1)$$

ensuring constant angular velocity.

When $t_{\text{rem}} \leq 0$ (the common case): the reference *snaps* to the target and the reference velocity is set to zero.

8.2 Velocity Mode — Lie-Group Integration

In velocity mode, the target provides a twist $\boldsymbol{\xi}_d$ in the TCP frame. The reference pose is integrated on SE(3):

Body-Frame Velocity Integration

$$T_{\text{ref}}(k{+}1) = T_{\text{ref}}(k) \cdot \text{Exp}(\boldsymbol{\xi}_d\, \Delta t) \tag{19}$$

where $\Delta t = 1/f_c$.

Why right-multiply?

The twist $\boldsymbol{\xi}_d$ is in the TCP (body) frame, so we right-multiply. This means "move forward along my current heading" regardless of the global orientation — exactly the right semantics for teleop or tool-frame velocity commands.

After integration, the pose is clamped to Cartesian limits (§9).

For joint velocity mode:

$$q_{\text{ref}}(k{+}1) = q_{\text{ref}}(k) + \dot{q}_d\, \Delta t$$

9 Cartesian and Joint Limit Clamping

9.1 Translation Clamping

Position targets are clamped element-wise to a bounding box:

$$p_i = \text{clamp}(p_i,\ p_{\min,i},\ p_{\max,i}), \qquad i \in \{x,\, y,\, z\}$$

9.2 Rotation Clamping via Quaternion Log-Map

Rotation limits are expressed in the tangent space relative to a reference quaternion q_{ref}:

1. Relative quaternion: $q_{\text{rel}} = q_{\text{target}} \cdot q_{\text{ref}}^{-1}$
2. Log-map: $\boldsymbol{\phi} = \text{Log}(q_{\text{rel}}) \in \mathbb{R}^3$
3. Clamp: $\phi_i = \text{clamp}(\phi_i,\ \phi_{\min,i},\ \phi_{\max,i})$
4. Reconstruct: $q_{\text{clamped}} = \text{Exp}(\boldsymbol{\phi}_{\text{clamped}}) \cdot q_{\text{ref}}$

Why log-map instead of Euler angles?

The log-map provides a singularity-free, minimal parameterisation of the relative rotation. Euler angles suffer from gimbal lock and ambiguous representations near $\pm 90°$ pitch.

9.3 Velocity Scaling

Velocity targets are *scaled* (not clamped) to preserve direction:

$$\dot{p}_{\text{scaled}} = s\,\dot{p}, \qquad s = \min_i \left(\frac{v_{\max,i}}{|\dot{p}_i|} \right) \leq 1$$

A single scaling factor across all translational axes preserves direction. For rotational velocity:

$$\|\boldsymbol{\omega}\| > \omega_{\max} \implies \boldsymbol{\omega}_{\text{scaled}} = \omega_{\max} \cdot \frac{\boldsymbol{\omega}}{\|\boldsymbol{\omega}\|}$$

9.4 Bisquare Soft-Margin Deceleration

When a soft margin $m > 0$ is configured, velocity is smoothly scaled to zero as position approaches a limit. The normalised penetration is:

$$d = \text{clamp}\left(\frac{\text{distance from margin boundary}}{m},\ 0,\ 1 \right)$$

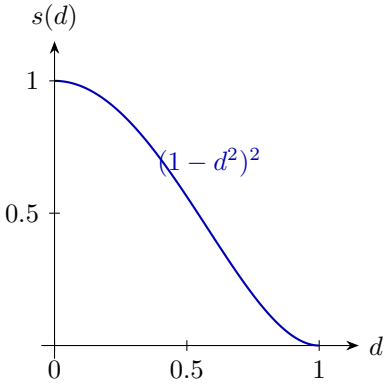
The scale factor uses the **bisquare** (biweight) kernel:

Bisquare Deceleration

$$s(d) = (1 - d^2)^2 \tag{20}$$

Properties:

- $s(0) = 1$: full speed at the margin boundary
- $s(1) = 0$: zero speed at the hard limit
- $s'(0) = 0$, $s'(1) = 0$: smooth onset/offset — no velocity discontinuities



10 Impedance Parameter Smoothing

10.1 Exponential Smoothing of Stiffness and Damping

Stiffness and damping are not applied instantaneously. An exponential moving average smooths transitions:

Exponential Smoothing

$$S_{n+1} = (1 - c)\, S_n + c\, S_{\text{target}} \tag{21}$$

where $c \in [0, 1]$ is the smoothing constant (separate values for stiffness and damping).

Reasoning

Instantaneous stiffness changes produce torque discontinuities that can excite structural resonances or jerk the arm. Smoothing acts as a first-order low-pass filter with effective time constant $\tau \approx \Delta t / c$.

Example

With $c = 0.05$ at 1 kHz, the time constant is $\tau = 1\,\text{ms}/0.05 = 20\,\text{ms}$. A step in stiffness from 100 to 500 N/m reaches 95% in $\approx 60\,\text{ms}$ ($\approx 3\tau$).

10.2 Feedforward Wrench Interpolation (Rate-Limited)

The feedforward wrench is ramped toward its target using a rate limiter:

$$F_{\text{ff},i}(k{+}1) = F_{\text{ff},i}(k) + \text{clamp}\big(F_{\text{target},i} - F_{\text{ff},i}(k),\ -\dot{F}_{\text{max},i}\,\Delta t,\ \dot{F}_{\text{max},i}\,\Delta t\big) \tag{22}$$

The target wrench is also clamped to configurable min/max bounds before ramping.

10.3 Force Feedback with Tare Offset

A feedback loop mixes the feedforward wrench with the sensed wrench:

$$F_{\text{total}} = F_{\text{ff}} + G_{\text{fb}} \odot \left(F_{\text{ff}} - F_{\text{sensed}}^{\text{tared}}\right) \tag{23}$$

where $G_{\text{fb}} \in \mathbb{R}^6$ are per-axis feedback gains and $F_{\text{sensed}}^{\text{tared}} = F_{\text{sensed}} - F_{\text{tare}}$.

The tare offset is captured by a service call and stored in the base frame. Each cycle it is transformed to the TCP frame:

$$F_{\text{tare}}^{\text{tip}} = R_{\text{tcp}}^{-1}\, F_{\text{tare}}^{\text{base}}$$

The total wrench (TCP frame) is then transformed to base frame for the impedance law:

$$F_{\text{ff}}^{\text{base}} = R_{\text{tcp}}\, F_{\text{total}}^{\text{tcp}}$$

(Applied separately to force and torque 3-vectors.)

11 Tracking Error Watchdog

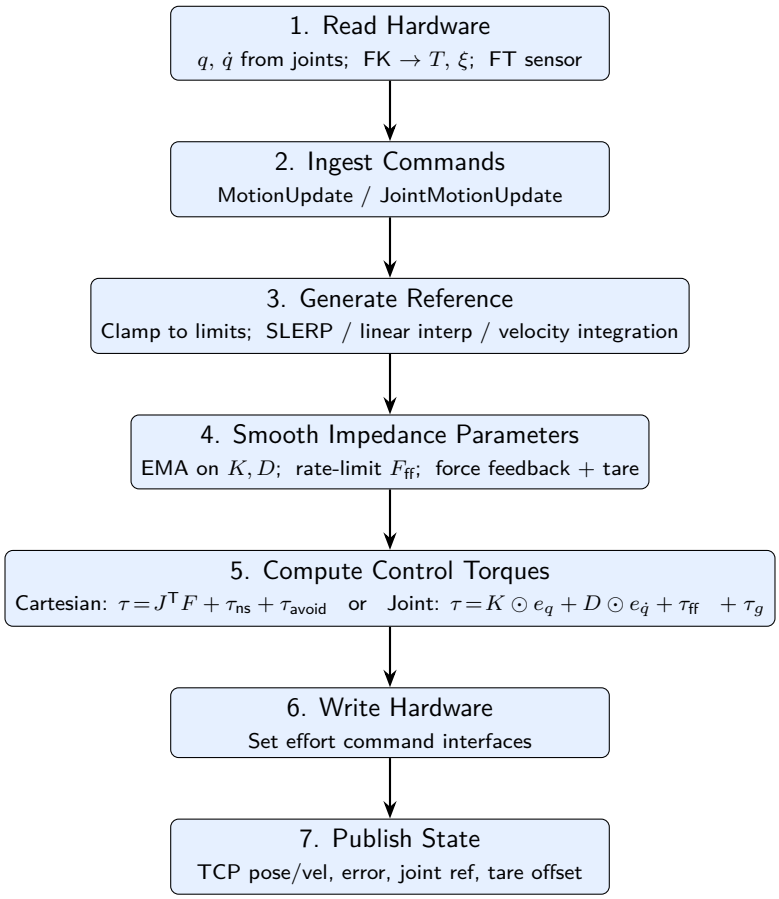
A safety mechanism detects when the controller cannot make progress:

1. Each cycle, compute $\Delta e = |e_x(k) - e_x(k{-}1)|$.

2. If $\|e_x\|_\infty >$ minimum error threshold **and** Δe is below minimum change threshold for longer than a configurable timeout $\Longrightarrow$ reset target to current pose.

This prevents indefinite build-up of integral error or feedforward wrench when blocked by obstacles, joint limits, or singularities.

12 Complete Algorithm Flow



12.1 Final Torque Equations

Cartesian impedance mode:

Complete Cartesian Impedance Torque

$$\begin{aligned}\tau \;=\; & \underbrace{J^{\mathsf{T}}\big(K\,e_x + D\,e_v + F_{\text{ff}} + K_I \odot e_I + F_{\text{offset}}\big)}_{\text{Cartesian impedance}} \\ 6pt] + \; & \underbrace{\left(I - JJ^{\dagger}\right)\left(K_{\text{ns}} \odot (q^*_{\text{ns}} - q) - D_{\text{ns}} \odot \dot{q}\right)}_{\text{Nullspace}} \\ 6pt] + \; & \underbrace{\tau_{\text{avoid}}}_{\text{Joint limit}} + \underbrace{\tau_g(q)}_{\text{Gravity}}\end{aligned}$$

Joint impedance mode:

Complete Joint Impedance Torque

$$\begin{aligned}\tau \;=\; & K \odot (q^* - q) + D \odot (\dot{q}^* - \dot{q}) \\ 4pt] + \; & \tau_{\text{ff}} + \tau_g(q)\end{aligned}$$

13 File-to-Concept Map

File	Sections
`src/actions/cartesian_impedance_action.cpp`	§3 (Wrench, J^{T}), §4 (Nullspace), §5 (Joint limits)
`include/.../cartesian_impedance_action.hpp`	§4.3 (Chiaverini pseudo-inverse docs)
`src/actions/joint_impedance_action.cpp`	§6 (Joint impedance)
`src/actions/gravity_compensation_action.cpp`	§7 (KDL gravity)
`src/aic_controller.cpp`	§8 (Reference gen), §9 (Clamping), §10 (Smoothing), §11 (Watchdog), §12 (Full flow)
`src/utils.cpp`	§2.2 (Exp/Log), §8.2 (SE3 integration)
`include/.../cartesian_limits.hpp`	§9.2 (Rotation limits)
`include/.../cartesian_state.hpp`	§2.1 (Pose representation)

References

[1] Chiaverini, S. (1997). *Singularity-robust task-priority redundancy resolution for real-time kinematic control of robot manipulators.* IEEE Transactions on Robotics and Automation, 13(3), 398–410.

[2] Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, Planning and Control.* Springer.

[3] Lynch, K. M. & Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control.* Cambridge University Press.